IslandUploads

Persistent data sites on HTTP, across partitionable islands

Project team

November 2, 2026

Contents

1 What this project is **1**
- 1.1 The idea . . . 1
- 1.2 The design in brief . . . 1
- 1.3 What it is for . . . 2
- 1.4 The tree . . . 4
- 1.5 Policy . . . 4
- 1.6 People and trust . . . 5
- 1.7 Reproducing everything . . . 6

2 How the design was reached **7**
- 2.1 Corrections that changed the specification . . . 7
- 2.2 Decisions and their reasons . . . 9

3 Requirements **12**
- 3.1 Tasks . . . 12
- 3.2 What must always, and never, happen . . . 12

4 Session types **14**
- 4.1 The door . . . 15

5 Across a partition **17**
- 5.1 Replication follows the peer's feed . . . 18
- 5.2 Lanes need a contiguous cursor . . . 19
- 5.3 Short files always open a lane . . . 19
- 5.4 A failing download yields its lane . . . 21
- 5.5 The feed needs the same order . . . 21
- 5.6 Concurrent policy edits . . . 21
- 5.7 What cannot be observed . . . 23

6 Implementation **24**

7 Open questions before one-shotting **25**
- 7.1 Security: settle before the source . . . 25
- 7.2 Policy . . . 26
- 7.3 Replication and storage . . . 27
- 7.4 Content, jobs and feeds . . . 27
- 7.5 Search and speed . . . 28
- 7.6 Subscribers . . . 28

7.7 Agreed, not yet in Lean 29
7.8 Limits of the tools 30

A The project Makefile **31**

Chapter 1

What this project is

1.1 The idea

HTTP is well specified for reading. `GET`, ranges, caching, content types and redirects work the same everywhere, and every browser and every tool agrees on them. Writing is another matter:

- nobody agrees on a permission system;
- there is little agreement on how uploads should work, and many options;
- authentication and authorization are a similar wild west.

Settle those, and you have persistent data sites to build on: something like a global SharePoint, built on standards. WebDAV was meant to be that, but its permission system was left unspecified; an attempt to build on Google's Go library for it failed for exactly that reason, and for lack of good hooks.

This project settles them. Writes are uploads in parts that resume; permissions are policies in the tree, evaluated from a verified token; identity is a set of seats, signed by issuers each site chooses to trust. Then a reverse proxy makes everything else usable from the files:

`/files/` data, and the apps that go with it. Static files are especially good for big data: videos, map layers. Replication helps with migration as much as with partitions.

`/apis/` services, preferably every service there is, so that apps point to them from static HTML. Whatever is about finding incrementally added bits, the indexed things, generally lives in a database behind an API.

1.2 The design in brief

This project restates, as a checked specification, the design of a file server that was once written by hand (`github.com/rfielding/microcms`, with per-file `--policy.rego` policies and AI-derived files), extended to shards on islands that may be partitioned.

People upload files on different islands. Every page shows its classification banner at the top and the bottom, and every file its own label. On every shard that lets them in, people see the same files, the same latest versions and the same banners, once replication has caught up.

That simple surface rests on a design that never waits for another island. The islands may be cut apart for a day, as when an anchor drags a cable across the Mediterranean; both sides keep accepting uploads and edits, and the shards reconcile when the link returns. Nothing is overwritten and nothing is lost.

Immutable records. A record is identified by `(id, dt)`, nanoseconds at the shard that wrote it. An edit is a new record with the same `id`, a later `dt`, and `prev_dts`: the versions it was based on. Two versions are concurrent when neither is reachable from the other.

Latest is a function. The latest version of a file is the one with the greatest `(dt, shard)`. Every shard holding the same records computes the same answer, and every superseded version stays readable as history.

Local visibility. `visible_dt` is the moment a record became readable on one shard. It is assigned on write and on import, never replicated, and a browser pages through a shard by it.

Anti-entropy. Each shard keeps a cursor per peer: how far up that peer's `visible_dt` feed it has fully imported. A shard that replicates from `a` and `b` keeps `{a: 50, b: 99}`, each value in that peer's own clock. It tells `a` "I have up to 50"; `a` answers that its feed now reaches 250, and the caller pulls 51 to 250 in `a`'s order. Records the peer imported from a third shard come along. A record arriving twice is stored once.

Content in parts. An upload starts empty, is appended to in parts small enough for any proxy, and is finished; the file exists only once its size and sha256 check out. A rejected part names the offset the shard holds, and the browser resends from there. Downloads and copies between shards use byte ranges, so a copy cut off by a partition resumes where it stopped.

Import lanes. A record from a peer becomes visible only once its blob has been pulled, by ranges, and verified, the way an upload is finished; that is when its `visible_dt` is assigned. Imports run in 128 lanes per shard and finish in any order; pulled records wait on disk for a lane and may lie far behind the cursor. Large blobs (over 64 MiB) take at most 96 lanes, so a short file always opens one. A download that fails gives up its lane, backs off, and later seeks to the offset it holds and continues by range. The cursor is kept in the peer's clock and moves only over a run of finished imports, while each import gets this shard's own `visible_dt` as it commits, so a shard turned off mid-replication misses nothing: delivery is at least once, a file may be sent again, and a repeat changes nothing. A visible record always has its blob, so a download never waits on another island.

Jobs make files. When a blob is created on a shard, that shard runs its jobs: `plans.pdf--image-001.jpg`, `plans.pdf--english.txt`, `plans.pdf--celebrities.json`. Outputs are ordinary files named after their source, so they have its permissions, and replicate like any other.

Policy. Walking from the root to the file, each level's policy overwrites entries of a table that ends as a decision such as `{list: true, read: true, write: false, banner: "SECRET//SQUIRREL", bg: "red", fg: "white", rank: 3}`; `list`, `read` and `write` are calculated by `Eval` from the caller's token. `list` decides whether the caller can even tell the file exists, `read` whether they may open it, `write` whether they may add a version; the banner is what they see. Job outputs answer to their source's policy.

1.3 What it is for

It is a content management system you can use with `curl`. Upload a folder into `/files/yeehaw/rjf`:

```
./concert/dimmuborgir.mp4
./concert/index.html
./concert/dimmuborgir.mp4--lyrics.vtt
```

The video is raw content; `index.html` is the viewer that goes with it; the `.vtt` is metadata about the video, timed text, named after it so it shares its permissions. Every uploaded file goes through a chain of AI jobs that make derived files out of raw ones: transcripts, extracted text, labels, translations. All text is indexed for search. Searching for "darkness" finds the

video, with the sentence around the word and how many seconds into the file it is, and links to `/files/yeehaw/rjf/concert/?ts=40`: the viewer, at forty seconds.

Content brings its own viewer. Map layers come with the map viewer that shows them; a dataset comes with the React app that explores it. The tree has two halves: `/files/` holds data and the apps that go with it, and `/apis/` holds services. Structured data is reached through a service, such as `GET /apis/.../db` with a JSON body, answering the same policies as files at that path. Users do this themselves: they write a custom app and upload it, or mount an API into the tree.

Mounting is what the `ingressHttps` interface is for. It is the one door from outside. It serves `/files/` itself and forwards each request under a mount in `/apis/` to an internal HTTP service, so the APIs in the backend mix with the apps and files in the frontend under one origin. A service lives in the island's internal scope, where nothing outside can reach it except through the door. At the door the token is verified and the walk is run for the request's path; the service receives the request with the caller's verified identity attached, and with any identity the caller tried to attach removed. The door is also where every request is counted: errors, latency and load, per mount.

An API is fast for many users at once the same way the record feed is: it serves its data as a feed, and each user reads it from their own cursor. The server keeps no state per user and answers every page the same way, so a thousand maps following the same tracks cost a thousand cheap page reads, not a thousand sessions. One idea runs through the whole system: records, replication, the UI and the APIs are all cursors over feeds.

The alternative was lived, too: one giant Python application, changed constantly to impress whoever was at the last meeting. Here the platform stays small and stable, and the apps that change with every meeting are the users' own files.

The aim is tons of React apps and tons of services, which people assemble from whatever they have access to. An app is just files. It runs as its viewer and is read-only to them: it sees exactly what its viewer may see, and changes nothing. Writes go through the upload endpoints and through APIs, which identify the user and check their permissions on every call, often looking up more than the token carries; the APIs are where security has to be airtight. That is what lets apps be written in a restricted environment, against data the developers may see, without a development team cleared for everything: the same app, uploaded into a classified tree, shows each viewer what that viewer is cleared for.

What a viewer of that demo is really thinking is security cameras. Upload the footage of many cameras and let the AI jobs write timed text over it: "a bomb goes off here", "a man places something here, the day before". A search then runs across every camera and every day at once, and each hit opens the right video at the right second. Immutable records with their sha256 suit evidence: nothing is overwritten, and every version of every annotation stays in history.

The same shape served a map. An earlier system, on one machine with two H200 GPUs, followed the war in Ukraine: feeds of COTS objects moving about, feeds of fixed objects everywhere, and news services that an LLM turned into reports of things to read and things to put on the map, each with its links. Here every feed is uploads, the LLM's reports and map layers are derived files, and the map is an uploaded viewer. Because a derived file is named after its source, every object on the map can answer "what is this?": clicking a tank rolling down Pennsylvania Avenue leads back to the feed entry, or the news item, that put it there.

That system also showed what this one must not repeat. It had no permission system at all. A query took ten seconds. And there was no way to forbid its developers from using public LLMs to write code, only a rule against Chinese models. Here permissions are the foundation; queries must be fast (a target is an open question); and since developers will use whatever tools they like, the design assumes it: apps are written outside, against data their writers may see, and only the

policies decide what an app shows.

1.4 The tree

Files live in a tree with ordinary URLs, so the server hosts content directly: `GET /files/centcom/j2-analyst/pla` returns the file, ranges and all. The first two levels are an organisation and a seat, such as `/files/centcom/j2-analyst`: seats are roles that people fill (below).

Folders are made the way `mkdir -p /files/pacom/billg/mswin` makes them: writing `/files/pacom/billg/m` creates every missing folder on the way, if the walk to the nearest existing folder gives the caller `write`. A new folder with no policy of its own simply adds no level to the walk, so its parent's table governs it, and later changes above reach it. If a folder's `id` is derived from its path, `mkdir -p` is idempotent: run on both islands while they are apart, it makes the same folders, and they merge after repair.

Whoever writes into an empty folder may supply its policy first: its `.--policy.rego` is then the folder's first file, written under the policy the walk finds above it. If they do not, the folder gets its parent's policy, by the same walk.

The first time a verified token says its bearer fills a seat, say `j2-analyst` of `centcom`, the shard creates `/files/centcom/j2-analyst/` and, as its first file, a `.--policy.rego` that lets whoever fills that seat write. From then on the seat manages its folder's policy, whoever is sitting in it.

1.5 Policy

A file's metadata is a table built by walking down the tree. It starts empty at the root, and at each level on the way to the file the entries that level sets overwrite the ones from above. For `/files/centcom/j2-analyst/plans.doc` the levels are

1. `/files/.--policy.rego`, then `/files/centcom/.--policy.rego`, then `/files/centcom/j2-analyst/.--p` each folder's policy, where it has one;
2. `/files/centcom/j2-analyst/plans.doc--policy.rego`, the file's own policy, last.

An entry is either supplied, written into the policy as it stands (a banner, its colours), or calculated by `Eval` from the caller's verified token (`list`, `read`, `write`). A level that says nothing about an entry leaves it as it came from above, so a folder can change its banner and keep its parent's rule for who reads. The table is computed from the root on every request, so a change to a folder's policy reaches everything below it at once.

Naming conventions settle the questions that usually start arguments here, such as whether directories are classified, or whether they are created implicitly. The defaults are:

Folders are implicit. Writing a file makes its folders, like `mkdir -p`.

Classify a folder or a file. A folder is classified by its `.--policy.rego`, which also governs the folder's own entry; a file by its `name--policy.rego`. Neither is required.

Metadata about a file is named after it. A file named `plans.doc--`*something* is about `plans.doc`, and by default has the same permissions: its walk passes through `plans.doc`'s table before its own. So whoever may read `plans.doc` may by default read `plans.doc--policy.rego`.

Restricting metadata is more of the same. `plans.doc--policy.rego--policy.rego` can keep readers of `plans.doc` from looking at its policy.

The shard runs every walk with its own authority. A caller who may not read a policy still gets its result: whether they may list, read and write the file, and the banner to show. A policy the caller may not list is answered like any missing file.

Names are content. A file's name can say as much as the file: a name that states an accusation about a named person tells everyone who sees it what the file holds, without opening it. So a name is governed by the same table as the content, and the permission that governs it is `list`: whether the caller may see that the file exists. Folders are the same: a folder's name, such as a seat's home folder, is governed by the folder's own table. Since a path is a chain of names, whoever may not list a folder may not learn the name of anything below it. Hence `list` only narrows on the way down: a lower level can take it away, never give it back. `read` implies `list`, since nobody opens what they cannot see. Other entries are overwritten, for now: whether `read`, `write` or the banner may loosen on the way down is an open question in the last chapter.

`list` without `read` is how people ask for access. The caller sees the name, its banner and that it is closed to them, and asks out of band: the system shows what exists and does not run the asking. A request for its content is refused with 403. Without `list`, the answer is 404, the same as for a path that never existed.

Clicking a file in the UI opens it in a viewer frame, a JPEG for instance, bannered top and bottom with `SECRET//SQUIRREL` from that file's table. The listing around it carries the page banner, the greatest rank among the files shown.

One flexible system. There is one classification system, flexible enough that the files it protects are heterogeneous. Medical records, GDPR personal data, foreign military releasability and national classifications have little in common: different markings, different rules, different reasons to open or close a file. Each is written as policy in the same tree and computed by the same walk, so one server holds them all side by side. The server interprets only a small core of the table (`list`, `read`, `write`, the banner, its colours and its rank); everything else a policy computes is carried through for the UI and for the policy below it.

The reason is a lesson. A product tied to one marking system, NATO's, had to be demonstrated to an audience in Amsterdam whose cases were GDPR, and the app had to be rewritten to accommodate them. Here no marking, rank table or rule lives in the source. A demo for a different audience is a different tree of policies, served by the same binary.

1.6 People and trust

Roles are seats. Files belong to roles, not to people. A role is a seat, and a person is placed in it; the token says which seats its bearer fills, and policies grant `list`, `read` and `write` to seats. The org chart is implicit in how roles are created: a boss's seat and a security officer's seat are granted access by the policies above the seats they oversee, so somebody can always still get to things. When people rotate out and move on, the identity provider empties their seat and fills it with someone else; the files never had a person's address as their owner, so nothing is orphaned, and nobody needs a full-time job of offboarding to make sure information is not lost.

How quickly someone loses a seat is how long their token lives: "how immediately do people get fired?" Shards verify tokens on their own, partitioned or not, so nothing reaches back to cancel one; it expires. A few hours suits some uses, and a tree can demand fresher tokens than another (an open question below).

Whom to trust. Tokens come from issuers, and different people trust different ones: one deployment trusts the USA and the EU, another China and Russia, another the USA, Canada and a local 7-Eleven. That is no stranger than a browser trusting many countries' certificate authorities, including countries that do not trust each other. A store clerk who checks a driver's licence can

issue a token that says only "over 21, resident of Virginia", with no card number, and any site that trusts that store can sell and ship age-restricted goods on it. Here, a policy sees the issuer beside the claims, so a folder can believe one issuer about age and another about clearance.

The same freedom applies to shards. Not every shard merges every other: each picks the peers it replicates from and the token issuers it trusts, and that choice is local, not replicated. The replication graph is whatever those choices make it. "The same view everywhere" is a promise to a user, not to the whole world: every shard that trusts the user's token enough to let them in at all shows them the same thing, once those shards hold the same records. A shard that does not trust their issuer is not one of their shards. The shards a user can use must therefore replicate with each other.

This is how separate file systems merge. Nothing on disk names a numeric user: ownership and access are claims in a signed token, evaluated by the policy found in the tree. A UID differs on every machine and ties a disk to a login; a path like `/files/centcom/j2-analyst` means the same thing on every shard, so two trees from two islands combine into one without remapping anyone.

1.7 Reproducing everything

From the LeanFM root, with the server running (`make serve`):

```
python3 scripts/chatui_specification.py $PWD/projects/IslandUploads \
  Project.Requirements.requirements Project.Implementation.implementation
lake build LeanFM.Session
lake env lean projects/IslandUploads/spec/Session.lean        # session checks
lake env lean --run projects/IslandUploads/spec/Islands.lean  # models
cd projects/IslandUploads
python3 scripts/mcp_islands.py && python3 scripts/mcp_report.py
make -C book
```

Chapter 2

How the design was reached

This book is where the design lives until it is written in Lean. The owner of the original system describes it as he built it. The description is written as requirements and small models, and the LeanFM MCP server checks the models. Every hole it finds is either a mistake in the transcription or a real choice in the design. Corrections go back into the specification, and the questions nobody can answer yet are kept in the last chapter. The source is written in one pass once the security questions there are settled; the others shape features that can ship later.

Each entry below says where it lives now. Once something is in a Lean file, that file is the record and this book only explains it.

2.1 Corrections that changed the specification

Each entry was caught in conversation and corrected the design. Each says where it lives: in a Lean file, or in this book only, until it is realized.

1. `visible_dt` **is local.** "visibleDt means nothing to us. That was just a field attached to the record to determine the order in which the current shard will read it." It is not replicated; an import gets the importing shard's own value. (`Requirements.proto`, field 15.)
2. **It is assigned at commit.** "That is when visibleDt is assigned, when it is committed and processed." Assigned at begin, the feed skips records, and the server shows how. (`Implementation.lean`, the writer.)
3. **Jobs run where the blob was created,** and everything they produce is a file: `plans.pdf-english.txt` (now `plans.pdf--english.txt`, below). Imports never run jobs. (`Requirements.lean`, proof "Jobs never run on imports".)
4. **Policy returns read and write.** `Eval(jwt, rego)` decides "whether they can even tell if the file exists". A file the caller may not read is answered exactly as a missing one. (`Requirements.lean`. Superseded below: existence is governed by `list`, and only a caller who may not list a file gets the missing-file answer.)
5. **The window is concurrency, not distance.** 128 lanes means 128 imports at once, finishing "in some random order"; a failing file may sit far behind the cursor. (`Requirements.lean`, "Imports are bounded".)
6. **Imports are downloads.** A record from a peer is pulled with its blob by ranges, resumed by seeking, and exists only when the blob is complete. (`Requirements.lean`, task `import`.)
7. **Replication is "how far up are you".** "You tell me how far up you are on replication, and I give you everything you have not seen." Marks per origin by `dt` became a cursor per peer in that peer's feed. (`Requirements.lean`; model `repl-cursor`.)

8. **Two clocks.** "peer.visibleDt is not our.visibleDt." The cursor is kept in the peer's clock, in its own table, and moves like a Kafka consumer's committed offset: imports finishing out of order hold it still, and the one that completes the run moves it to the top of the finished, contiguous run. (`Implementation.lean`; model `lanes-contiguous`.)
9. **Policies are found by walking the tree.** A file's own `plans.doc--policy.rego`, else its folder's `.--policy.rego`, else the parent's, up to the root. Files have ordinary URLs under `/files/<org>/<user>`, and a user's home folder and its policy are made the first time their token is seen. (Superseded below: the walk builds a table rather than stopping at the nearest policy, and home folders belong to seats. `Requirements.lean` still names `plans.pdf.rego` and serves content by `id` and `dt`.)
10. **A folder's policy comes first, or its parent's.** "If you write into an empty directory, supply a policy first! Or else, it gets its parent." Folders are made like `mkdir -p`: writing a file creates the missing folders on its path. (Book only.)
11. **Policy is a table built from the root down.** "A hashtable that is populated from the root, with entries overwritten as you go down the tree to the file." Not the nearest policy alone: each level overwrites only the entries it sets, supplied ones like the banner and calculated ones like `read` and `write`. "You might have access to a file, but not the policy about it": the shard runs the walk with its own authority and the caller sees only the result. (Book only: `Requirements.lean` still has one `.rego` per file or folder.)
12. **Naming conventions instead of arguments.** "Should directories be classified? Are directories implicitly created? We want sensible defaults, where you may put a classification over a directory or on an individual file, and use naming conventions to get out of these weird pickles." Folders are implicit; a folder or a file may carry a policy; `name--`x is metadata about `name` with its permissions by default, and `name--policy.rego--policy.rego` restricts who sees the policy. (Book only.)
13. **Names are content.** From an example file whose name alone accused a named person of a crime: "I wonder what is in it?" A name is governed like the content, so no channel may carry a name its reader may not see, and the permission over names narrows down the path. (That permission became `list` in the next entry. Book only.)
14. **`list` is its own permission.** "Another permission: List, which can be granted so that you can see that a file exists, and ask for permission. Different classification systems (medical, GDPR, foreign military) can be wildly different." And: "It is one flexible classification system, such that the files protected are heterogeneous." The lesson behind it: a product tied to NATO markings had to be rewritten to demonstrate GDPR cases to an audience in Amsterdam. The name is governed by `list`, the content by `read`; the server interprets only a core of the table. (Book only.)
15. **Roles are seats.** "Roles are like seats, into which butts are placed. Better to assign files to a role, that a person fills. That way, when people rotate out and move on, you don't need a full-time offboarding guy to make sure information is not lost." Asking for access is out of band; the org chart is implicit in how roles are created, and the token setup ensures that a boss or a security officer can still get to things. Home folders are seats too; token life is "how immediately do people get fired?", a few hours for some uses. (Book only.)
16. **Many issuers.** "Imagine that you trust JWT CAs: USA, EU. Somebody else trusts CN, RU. Someone else trusts US, CA, 711." A token from a local store that checked a driver's licence can say "over 21, Virginia" and nothing more. "You will not necessarily merge every shard. But you are certainly free to pick shards and JWT signers that you trust." And: "The shards you have should be the same everywhere, given that the shard trusts your cert enough to use the system at all." (Book only.)

17. **The point: a CMS you use with `curl`.** "Subject all uploaded files to an AI chain that makes derived files out of raw files. If you want to upload map layers, upload the viewer that goes around it. The idea is tons of React apps, tons of services. People assemble apps out of everything they have access to, and the app development can be done in the restricted environment, without necessarily needing a dev team that is cleared for anything." Search hits carry their context and their position, such as forty seconds into a video. And what the audience really thinks: "upload a whole bunch of security cameras; let AIs put vtt on it". (Book only.)
18. **Apps read, APIs write.** "Usually app viewers are read-only to its users. APIs use the user's id to look up what their permissions actually are, not necessarily all in the JWT. The APIs need to be pretty airtight on security." (Book only.)
19. **A map, too.** An earlier map system: feeds of moving and fixed objects, news turned by an LLM into reports with links and things on the map, and a "what is this?" on every object. And its lessons: "no permission system; 10 s query latency; no way to tell people 'no public LLMs to code'; but we did have a 'no Chinese models' rule." Twitter feeds were a source too, and trying that API for a month cost $2000 by surprise: an external feed needs a hard budget. A process that pretended to be an airport, feeding out track updates to make planes fly around, stress-tested the map. (Book only.)
20. **Cursors over API feeds.** "Put stuff behind APIs, like /apis/, and /files/: the apps, and the data plus apps. Allow users to do some of this stuff themselves. Fast APIs meant supporting multiple users using cursors over data feeds coming out of the APIs. Instead, we had a giant Python app that was constantly updated to impress whoever it was we last had a meeting with." (Book only.)
21. **The door forwards.** "That is what the ingressHttps is for: it forwards requests to internal http, to mix and match the APIs in the backend with the mounted React apps and files in the frontend." Its global type: browser to door `GET /apis/$x/$task`, door to `host[$x] GET /$task`, like the reverse proxy in microCMS; with big streams and websockets. (Protocols in `Session.lean`: `Forward`, `Socket`; its negative requirements are book only.)

2.2 Decisions and their reasons

Decision	Why	Lives in
Writes never wait for another island	Islands get cut apart for a day: "somebody always dragged an anchor across the Mediterranean." A file server that stops taking uploads is worse than one that reconciles later.	Requirements; model `repl-ordered` (writes commit while apart)
Records are immutable: `(id, dt)` with `prev_dts`	The original MongoDB design. Concurrent versions are kept, never overwritten, so nothing is lost in a merge.	Requirements; proto
Latest is the greatest `(dt, shard)`	A function of the records held, so every shard holding the same records shows the same file.	Requirements

Decision	Why	Lives in
Mutable fields are local	"Immutable with exceptions to keep the schema simple": one record type, with `visible_dt` ignored when imported.	Proto
A cursor per peer, in the peer's feed	A late commit lands at the end of the peer's feed, past every cursor already given out. No origin needs to commit in `dt` order, and records from a third shard are relayed.	Requirements; `repl-cursor` proved
Delivery at least once	Exactly once needs coordination the islands cannot afford. "It's ok if we get a bunch of files that exist and are the right size; we just have to cursor over them."	Requirements; Implementation
Blob before record	"Pull the blob in a task that makes the record exist." A visible record always has its blob, so a download never waits on another island. After an outage the network may be "going crazy catching up", so pulls are bounded rather than greedy.	Requirements
128 lanes, 32 kept for small files	Bounded concurrency handles "incredible bursts", and a short file always opens a lane.	Requirements; `sched-reserved` proved
A failing import yields its lane	Otherwise "it would effectively monopolize one lane as long as it tries".	Requirements; `retry-yield` proved
Uploads in parts	Proxies have body-size limits: "make a zero length file, then keep appending to it with different http calls, then tell it you are done."	Requirements; Implementation
Ranges are mandatory	"Even if you do not ask for a range, sometimes the browser will offer a range." Seeking in a 9000 MB video was instant because of it.	Requirements; Implementation (`ServeContent`)
Encryption is the volume's	The original wrote seekable ciphertext by hand: "unimaginable pain; I do not need to do that again." With LUKS or the cloud disk, a range stays a plain file read.	Implementation
Pure-Go SQLite	The original's Postgres cgo driver "made build times impossibly long". `modernc.org/sqlite` keeps the original indexes.	Implementation

Decision	Why	Lives in
Policy as `Eval(jwt, rego)`	No marking system is built in, so a GDPR demo needs no rewrite of a NATO product. The banner is computed per caller: {`list, read, write, banner, bg, fg, rank`}. The page shows the greatest rank at top and bottom.	Requirements; proto
Concurrent policies must all allow	With latest-wins, a later edit on the other island undoes a tightening.	Requirements; `policy-allHeads` proved
One session per page	A peer that must know when the puller stops is a protocol bug; each page is answered on its own.	`Session.lean`

Chapter 3

Requirements

`spec/Requirements.lean` and `spec/Requirements.proto` are the contract; this chapter reads them as they are today. Where the design in the first chapter has moved on (`list`, the tree walk, seats, the door), the last chapter lists what has yet to reach them. Four roles take part: `Browser`, `Shard`, `Peer` (the same shard process, answering another shard) and `Worker` (the shard's job runner).

3.1 Tasks

`upload` start, parts, finish, answered by the shard alone; new content for an existing file is an upload naming the file and the versions it is based on. `request_id` makes a retried start idempotent.

`update` a version without new content (rename, delete); never refused because its base is not the latest, only when the shard does not hold the base at all.

`feed` the next page after the browser's last `visible_dt`, filtered by policy, each record with its decision and the page with its banner.

`history` every version of one file.

`download` checked against policy and served from the local blob: 200 for the whole file, 206 for any range, which browsers send on their own when a video seeks, and 416 for a range outside the file.

`replicate` one `IHave` exchange, then pages of the peer's feed past the caller's cursor, in the peer's `visible_dt` order.

`import` a record's blob pulled by ranges in a lane, resuming after a timeout; the record becomes visible at the end.

`derive` a job on a blob created here; its outputs become files.

3.2 What must always, and never, happen

- Writes never wait for another island; a file exists only when complete; parts and copies resume from the offset already held; ranges are exact.
- Records never change between shards; every superseded version stays in history.
- Cursors are prefixes: a shard holds every record of a peer's feed up to its cursor for that peer.
- The feed never skips a record it can show; every shown file carries its label; the page banner is the most restrictive label shown, at top and bottom.
- Two shards holding the same records give the same token the same page.

- Nothing is shown that policy denies, and a file the caller may not read is answered exactly as a missing one; writes need `write`; job outputs are governed by their source; unverifiable tokens are denied; concurrent policy versions are combined, not overwritten.
- A visible record has its blob; imports are bounded; cursors advance only over finished imports; jobs run only where the blob was created.
- Possibly: the islands are apart for a day. Handling: keep writing on both sides, retry anti-entropy on its timer, records first, then blobs.

Chapter 4

Session types

`spec/Session.lean` writes the tasks as global types with the `LeanFM.Session` library. Four of them:

```
protocol Upload (Browser, Shard) {
  u: UploadStart from Browser to Shard;
  choice at Shard {
    o: UploadStarted from Shard to Browser cause u where o.requestId = u.requestId /\ o.offset = 0;
    rec Parts {
      choice at Browser {
        p: UploadPart from Browser to Shard;
        choice at Shard {
          a: UploadPartAck from Shard to Browser cause p where a.offset > p.offset;
          continue Parts;
        } or {
          x: UploadRejected from Shard to Browser cause p where x.status = 409;
          continue Parts;
        }
      } or {
        f: UploadFinish from Browser to Shard;
        choice at Shard {
          r: UploadResponse from Shard to Browser cause f where r.requestId = u.requestId;
        } or {
          UploadRejected from Shard to Browser cause f;
        }
      }
    }
  } or {
    UploadRejected from Shard to Browser cause u;
  }
}

protocol Download (Browser, Shard) {
  d: DownloadRequest from Browser to Shard;
  choice at Shard {
    a: AccessDenied from Shard to Browser cause d where a.status = 401 \/ a.status = 404;
  } or {
    DownloadResponse from Shard to Browser cause d;
  } or {
    RangeNotSatisfiable from Shard to Browser cause d;
```

```
  }
}

protocol Replicate (Shard, Peer) {
  i: IHaveRequest from Shard to Peer;
     IHaveResponse from Peer to Shard cause i;
}

protocol PullPage (Shard, Peer) {
  q: RecordsRequest from Shard to Peer;
  p: RecordsResponse from Peer to Shard cause q where p.lastVisibleDt >= q.afterVisibleDt;
}
```

Its `#guard` lines check, whenever the file is elaborated:

- every protocol is well formed, and every role can tell which branch it is in;
- every message on an HTTP channel is a request its sender may make, or the response to a pending request, going back to the role that asked: no cuts;
- every channel's conditions (timeouts, retried requests, rejection, closes) are handled;
- browsers reach shards only through ingress, shards each other through peering, and a worker only from its own machine;
- the shard-to-shard channel between `us` and `eu` also surfaces `partition`, which the protocols do not handle: the design handles it instead, and the next chapter checks how.

Two choices in these types are worth a word. In `Forward` (below) the host is contacted in one branch only; it need not be told about the others, because its part of the session starts with the request it receives, as a thread forked for that session. And anti-entropy is not one long session but one per page: the peer answers each `RecordsRequest` on its own, and whether to ask again is the puller's decision. A peer that had to know when the puller stopped would be a protocol bug; the checker reported exactly that for a first draft that wrapped the pages in a loop.

4.1 The door

A browser never reaches a mounted service itself. `GET /apis/$x/$task` goes to the shard's `ingressHttps`, which verifies the token, runs the walk for the path, and forwards `GET /$task` to the host mounted at `$x`; the answer goes back the way it came, as the reverse proxy in microCMS did. A websocket opens the same way and then carries frames both ways, each direction until its own end closes it. Big responses stream: one response whose body arrives over time, on a channel whose `close` is handled.

```
protocol Forward (Browser, Door, Host) {
  r: ApiRequest from Browser to Door;
  choice at Door {
    f: ApiForward from Door to Host cause r where f.task = r.task;
    a: ApiReply from Host to Door cause f;
    p: ApiResponse from Door to Browser cause r where p.status = a.status;
  } or {
    d: AccessDenied from Door to Browser cause r where d.status = 401 \/ d.status = 404;
  }
}
```

```
protocol Socket (Browser, Door, Host) {
  u: WsUpgrade from Browser to Door;
  choice at Door {
    g: WsUpgrade from Door to Host cause u where g.mount = u.mount;
    a: WsAccepted from Host to Door cause g;
    WsAccepted from Door to Browser cause u;
    par {
      rec Up {
        choice at Browser {
          FrameUp from Browser to Door;
          FrameUp from Door to Host;
          continue Up;
        } or {
          CloseUp from Browser to Door;
          CloseUp from Door to Host;
        }
      }
    } and {
      rec Down {
        choice at Host {
          FrameDown from Host to Door;
          FrameDown from Door to Browser;
          continue Down;
        } or {
          CloseDown from Host to Door;
          CloseDown from Door to Browser;
        }
      }
    }
  } or {
    d: AccessDenied from Door to Browser cause u where d.status = 401 \/ d.status = 404;
  }
}
```

`Session.lean` checks both. The forwarded call passes the HTTP discipline: every message is a request its sender may make (browser to door, door to host) or the answer to one, going back to whoever asked. The host is in the island's internal scope: the door reaches it, a browser and the other island cannot. And the checks bite: a variant in which the host answers the browser directly is reported as a cut, since the browser never called the host; and the websocket fails the HTTP check, which is why, after its upgrade, it is checked as a stream.

Big deployments of the same shape run on Kubernetes: the door is an ingress controller, a mount is a service, and the internal scope is the cluster network.

Chapter 5

Across a partition

`spec/Islands.lean` builds seven small models of shards, lanes, feeds and policies, each in variants, and `scripts/mcp_islands.py` asks the LeanFM MCP server about them. Every answer is in `reports/mcp/results.json`.

model	question	verdict	evidence
`feed-byDt`	the feed never skips a readable record	**disproved**	counterexample: `BeginA1, BeginA2, CommitA2, FeedPage, CommitA1, Missed`
`feed-visibleAtBegin`	the feed never skips a readable record	**disproved**	counterexample: `BeginA1, BeginA2, CommitA2, FeedPage, CommitA1, Missed`
`feed-visibleAtCommit`	the feed never skips a readable record	**proved**	none, over all 76 nodes
`lanes-contiguous`	from every state the shards can still converge	**proved**	none, over all 59 nodes
`lanes-contiguous`	imports finish in reverse order: the cursor stays at 0, then the last to finish moves it to 3	**proved**	witness: `CommitA1, CommitA2, CommitA3, PullAtoB, ImportA3to0, ImportA2to0, ImportA1to3`
`lanes-max`	from every state the shards can still converge	**disproved**	counterexample: `CommitA1, CommitA2, PullAtoB, ImportA2to2, RestartB`
`policy-allHeads`	no read is allowed that a held concurrent policy denies	**proved**	none, over all 137 nodes
`policy-allHeads`	eu can read under the policy it holds	**proved**	witness: `ReadB`
`policy-latestWins`	no read is allowed that a held concurrent policy denies	**disproved**	counterexample: `TightenOnA, EditOnB, PullAtoB, LeakB`
`repl-cursor`	from every state the shards can still converge	**proved**	none, over all 1108 nodes
`repl-ordered`	from every state the shards can still converge	**proved**	none, over all 291 nodes

model	question	verdict	evidence
`repl-ordered`	the shards must converge	**disproved**	counterexample: `BeginA1, BeginA2, BeginB1, CommitA1, CommitA2, CommitB1, PullAtoB, then around again`
`repl-ordered`	a write commits while the islands are apart	**proved**	witness: `Partition`
`repl-unordered`	from every state the shards can still converge	**disproved**	counterexample: `BeginA1, BeginA2, CommitA2, PullAtoB`
`retry-hold`	an import that can succeed gets a lane while others keep failing	**disproved**	counterexample: `StartF1, StartF2, RetryF1, then around again`
`retry-yield`	it also finishes, without assuming fair delivery	**disproved**	counterexample: `StartF1, StartF2, FailF1, StartX, FailF2, BackoffF1, StartF1, FailF1, then around again`
`retry-yield`	an import that can succeed gets a lane while others keep failing	**proved**	none, over all 41 nodes
`sched-reserved`	a small import starts before any large one finishes	**proved**	none, over all 66 nodes
`sched-shared`	a small import starts before any large one finishes	**disproved**	counterexample: `StartB1, StartB2, DoneB1`

5.1 Replication follows the peer's feed

The first model pulls by a mark per origin: everything of that origin with a `dt` above the mark. With one writer per shard, records of an origin become readable in `dt` order, and from every state the shards can still converge (over all 291 nodes). Allow two concurrent writers and the server finds the hole in four steps: `a2` is committed before `a1`, `b` pulls and moves its mark past `a2`, and `a1` is never pulled.

The design does not ask origins to commit in `dt` order. A shard asks "how far up are you?" in the peer's own feed: its cursor is a position in the order the peer made records readable, which is `visible_dt`, assigned at commit. A record committed late lands at the end of that feed, past every cursor already given out, so nobody's cursor can skip it. With the same two concurrent writers, and the cursor in place of the mark, the server proves that from every state the shards can still converge (over all 1108 nodes). This is the feed result below, applied between shards instead of to a browser.

Writes do commit during a partition (the witness starts with `Partition`). Convergence must still be helped along: without fairness the server returns a lasso in which `b` keeps pulling and `a` never does.

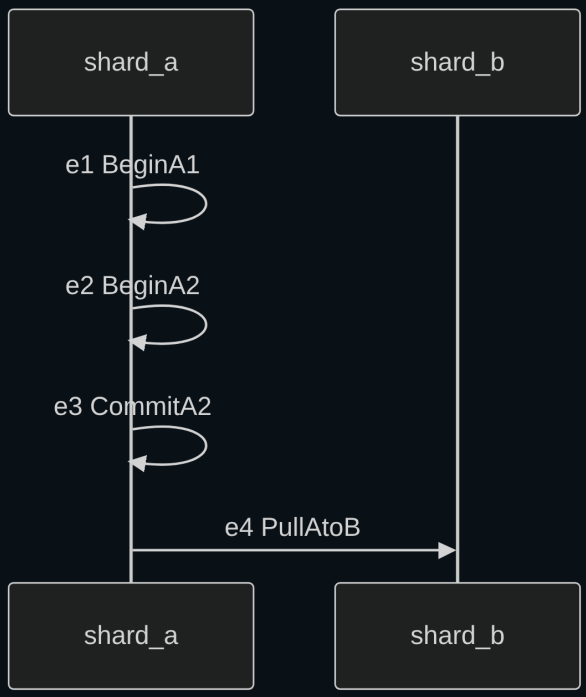


Figure 5.1: Marks by `dt` with commit order broken: `b`'s mark passes `a1` before `a1` is readable. Drawn by `check_trace` from the server's counterexample.

5.2 Lanes need a contiguous cursor

The cursor is kept in the peer's clock, not ours. Each import is given this shard's own `visible_dt` when it commits, and that says nothing about how far up the peer's feed we are, so the cursor cannot be read back from local records. It is the committed offset of a Kafka consumer: a separate value per peer, moved by the imports, in the same commit as the import that moves it.

Import lanes finish out of order. If the cursor moves to the highest position imported, a restart that loses the queue loses a record for good: `a2` finishes first, the cursor jumps to 2 past `a1`, the restart drops `a1`, and the next pull starts after `a2`.

The fix is to remember which pulled records are still in flight and move the cursor only to the highest peer value below which every one has finished. If a page's imports finish in reverse order, nothing moves while they do; when the last one, the lowest, finishes, the cursor jumps straight to the top of the page. The server proves that this converges from every state (over all 59 nodes, three records), and finds the reverse-order run itself.

Turn a machine off anywhere in this and the next round asks again from the cursor. Records above it that had already finished are resent; each is already held by its `(id, dt)` and is dropped without pulling its blob again. At worst a file in flight is copied twice.

5.3 Short files always open a lane

With every lane open to every import, large files can fill them all, and a small file waits for one of them to finish. Keeping a share of the lanes for small files means a small import starts before any large one finishes (scaled down to two lanes, one of them reserved, over all 66 nodes).

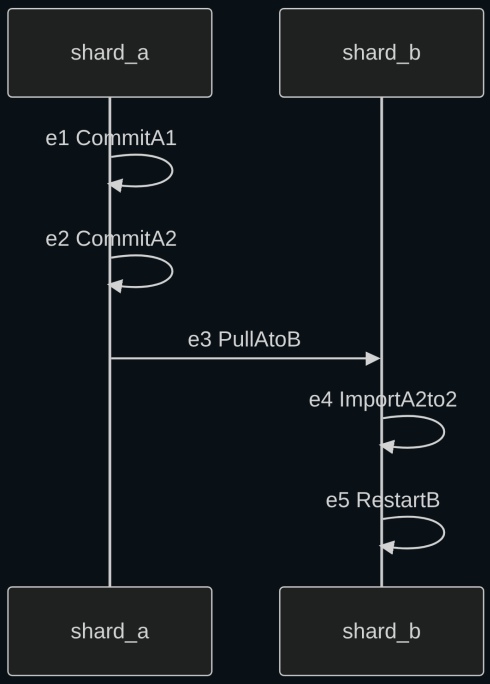


Figure 5.2: Cursor at the highest import: `ImportA2to2`, then `RestartB`, and `a1` is never pulled again. Each import is labelled with the cursor after its commit.

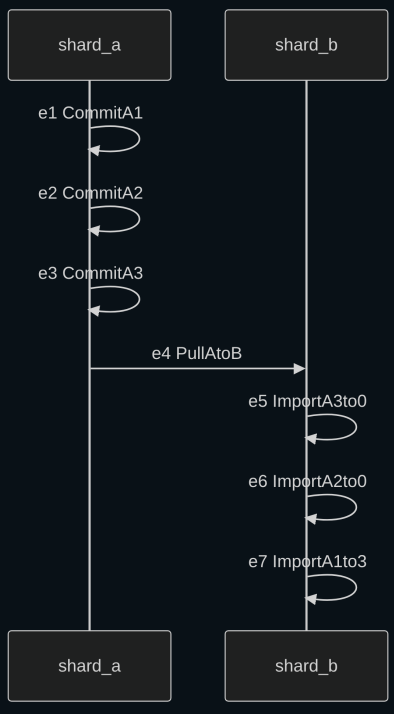


Figure 5.3: A contiguous cursor, imports finishing in reverse order: `ImportA3to0` and `ImportA2to0` leave it at 0, and `ImportA1to3` moves it to 3.

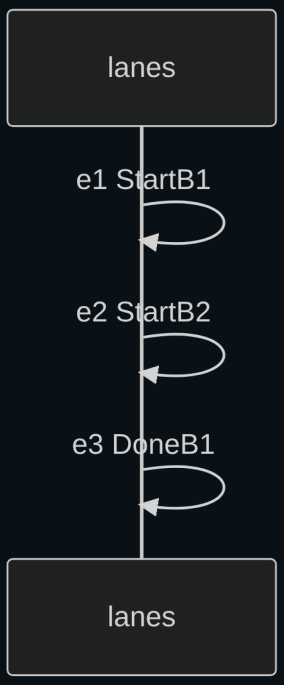


Figure 5.4: Shared lanes: two large imports take both lanes, and one finishes before the small file starts.

5.4 A failing download yields its lane

A blob that keeps failing to download would hold its lane for as long as it retries, and two such blobs hold both lanes of the scaled-down model: an import that would succeed never starts. If a failed attempt gives up its lane and waits out a backoff, that import always gets a lane (over all 41 nodes).

Getting a lane is all that yielding can promise. Whether `X` then finishes depends on the network delivering its bytes, and the server shows a run in which it never does while the failing imports cycle. That is the fair-delivery assumption again, stated as a requirement rather than hidden.

5.5 The feed needs the same order

Paging by `dt` misses records, and so does paging by a `visible_dt` taken when a write begins: a later write commits first, the browser's cursor passes the earlier one, which then becomes readable behind it. Assigning `visible_dt` when a write commits, through one writer, is proved complete.

All three holes have one cause: a cursor or a mark moved past something not yet readable. The controls are the single writer queue (records become readable in the order others page by, browsers and peers alike) and the contiguous cursor; their negative tests are these counterexamples.

5.6 Concurrent policy edits

`us` tightens `plans.pdf.rego` while `eu` edits it later without seeing the tightening. If the latest `dt` wins, the edit on `eu` reopens the file once the two are combined:

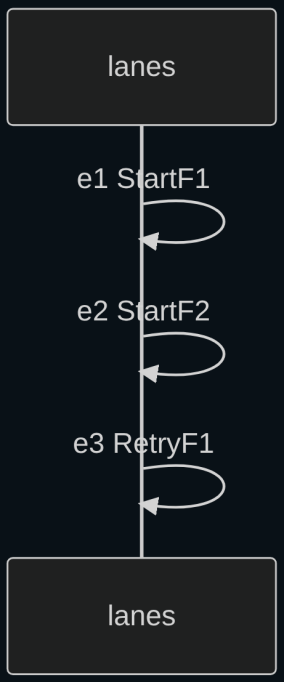


Figure 5.5: Retrying in place: `F1` and `F2` keep failing in the two lanes, and `X` never starts.

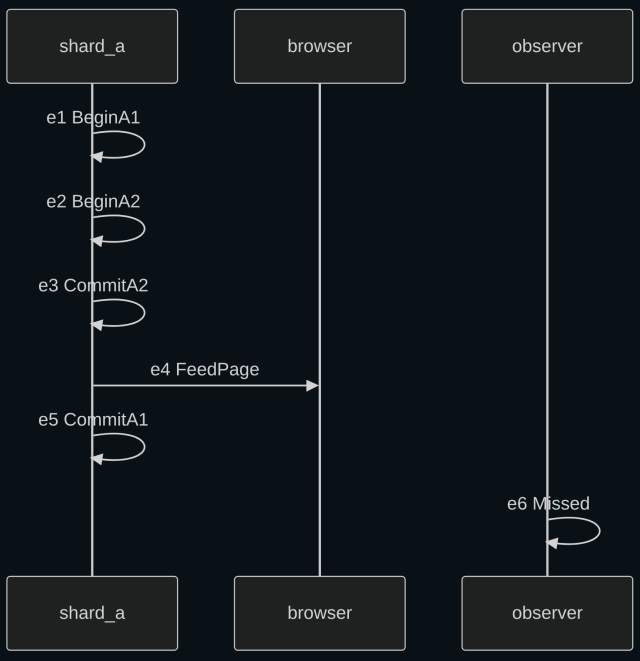


Figure 5.6: `visible_dt` assigned at begin: `A2` commits first, the page moves the cursor past `A1`, and `A1` is missed.

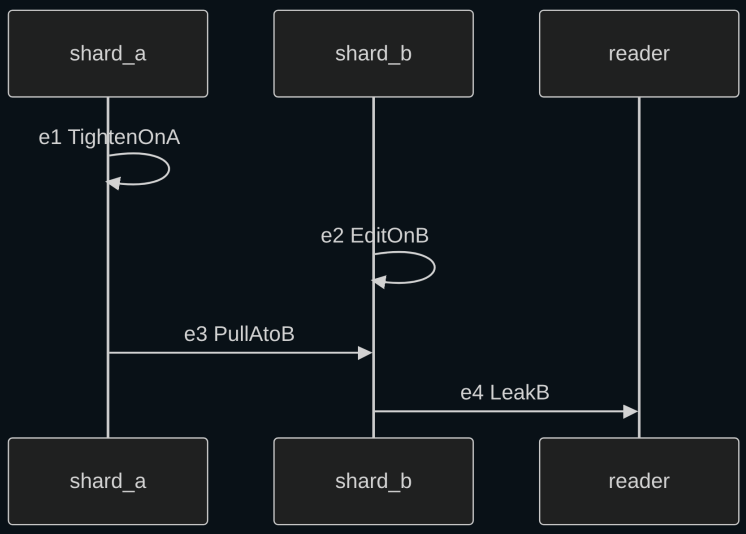


Figure 5.7: Latest wins: `b` holds the tightening and the later edit, and lets the reader in.

Requiring every concurrent version to allow, until a version based on all of them is written, is proved leak-free. One thing remains, by design: during the partition, `eu` enforces the policy it holds. A shard cannot enforce a tightening it has not received. If that is unacceptable for some classification, changes to its policy must wait for a quorum, which is a different requirement.

5.7 What cannot be observed

implementation-only visible_dt is assigned in the order records become readable

implementation-only a shard lets a caller read a file while it holds a concurrent version of the governing policy that denies that caller

derived LeakA and LeakB never happen

The first two must be restated over events before they can be tested; the third is how this chapter states them.

Chapter 6

Implementation

`spec/Implementation.lean` maps every role and message:

- one Go binary, `islandd`, per shard; records in SQLite with the original collection's indexes: `(dt)`, `(parent_id, id, dt)`, `(id, dt)`, `(shard, dt)`, `(visible_dt)`;
- one writer goroutine per shard assigns `visible_dt` and commits in the same step: the control for both commit-order holes;
- strictly increasing nanosecond timestamps per process;
- uploads as `POST /uploads`, `PUT /uploads/{id}` with an `Upload-Offset` header, `POST /uploads/{id}/fini` parts are appended to a file and fsynced, and the file is renamed into place only after its sha256 checks; downloads and peer copies answer `Range` with 206;
- encryption at rest is the volume's (LUKS or the cloud disk's), not the application's, so a range stays a plain file read;
- anti-entropy every 5 seconds per peer, while the import queue has room; 128 lane goroutines pull blobs by ranges into part files, verify, rename, and hand the record to the writer; each peer's cursor, in that peer's clock, moves in the commit that finishes a contiguous run of imports;
- downloads through `net/http ServeContent`: 200, 206, `If-Range`, `HEAD` and 416, with the stored content type;
- JWTs verified before any read or write; policies evaluated by embedded OPA with network and file builtins disabled, a 50 ms budget, errors denying;
- jobs as configured commands, two workers per shard, outputs written back as files.

Chapter 7

Open questions before one-shotting

The specification checks, and its risky parts are checked by the server. What follows is everything not yet in a Lean file: questions the source would otherwise answer by guessing, requirements agreed but not yet written, and the limits of the tools. Each item moves out of this chapter when it is realized.

The questions are grouped by what they hold up. The first group decides security and must be settled before any source is written; the others shape features that could ship later.

7.1 Security: settle before the source

1. **Keeping apps read-only.** An app runs in its viewer's browser, so whatever lets it read could let it write. The proposal: serve uploaded content from its own origin, whose session can only read, under a Content Security Policy that lets it reach nothing but this server; writes need credentials that origin never holds. Is that enough, or should each folder's apps get an origin of their own, so one app cannot read what another folder's app was shown?

2. **Who may write a policy?** Anyone who may write a folder can create `plans.doc--policy.rego` next to someone else's `plans.doc` and, being the last level of the walk, it overwrites what it likes. Creating or editing a `--policy.rego` should need `write` under the policy it would replace, and perhaps a separate decision: should `Eval` return `admin` beside `list`, `read` and `write`?

3. **May a lower level loosen?** `list` may not, because names are content. May `read`, below a folder the caller may list? May `write`? May a lower banner replace a higher one, or is the banner the greatest rank on the path?

4. **The root.** The walk starts at `/files` (and at `/apis`). What do their policies say, and who may write them? With no policy found at all, the answer is deny.

5. **A policy change reaches back in time.** A subscriber's cursor moves past records it may not read. If a later version of their policy lets the caller in, those records are already behind the cursor and are never sent; if it shuts the caller out, files already shown stay on screen. The new policy version is a record in the feed, but the subscriber may not be allowed to read it, so it cannot be the cue. The shard has to act: when a policy version commits, it evaluates the files below it again and sends the changed decisions as fresh events, appearing and withdrawing files. How is that bounded when the policy governs a large tree?

6. **Records relayed from an untrusted origin.** A peer's feed carries records it imported from third shards. If a shard trusts the peer but not the third shard, does it drop those records? A policy file from an origin it does not trust must not govern anything there. And so that a relay cannot forge an origin, should each record be signed by its origin shard?

7. **Attributes beyond the token.** APIs look users up by id, because not every permission is in the token. Where does that directory live, and what happens during a partition? If two shards hold different directory data, one token gets different answers on each, so the directory needs the same treatment as the records: replicated, and its answers as of the same records.

8. **Paths.** Is everything under `/files/` or `/apis/`? The upload endpoints (`POST /uploads`, `PUT /uploads/{id}`) sit outside both today; should an upload be a `PUT` to the file's own path under `/files/`, in parts? Names must not contain `/`, must not be `.` or `..`, and a name ending in `--policy.rego` is always a policy. The proposal for symlinks and relative paths, which seat folders tend to raise: none in the store; paths are canonical and `..` is refused, because a second name for a file would let a link grant access; "my files" across the seats a person fills is a view the UI builds from the seats in their token.

9. **Mounting.** A mount is an internal HTTP address that `ingressHttps` forwards to. How is it declared: a file in the tree, such as `/apis/db/.--mount.json`, written under the policy at that path, so mounts replicate like everything else? Does each island run its own copy of a service, or forward across islands? And how many requests may one mount have in flight, so that one slow service cannot take every connection the door has, the way a failing import could once take every lane?

10. **The cursor can reveal hidden activity.** When a page holds only records the caller may not read, the cursor still moves, which tells the caller that something they cannot see was written. Is that acceptable, or should the cursor advance only on a schedule, whatever was written?

7.2 Policy

1. **What does `Eval` see?** Does each level's policy get the table from above as input, so it can decide relative to it ("as my parent, but not for contractors")?

2. **Which entries are supplied?** Banner and colours are; what else (retention, content type, job list, allowed models)?

3. **Concurrent versions of one level's policy** are combined entry by entry: `list`, `read` and `write` by and, the banner by greatest rank. What about other entries?

4. **One rank for heterogeneous files.** A listing may show a medical record beside a national marking, and its page banner is the greatest rank among them. That needs every policy's rank on one scale. Who defines the scale: the root policy, or a supplied entry every policy must use? And what does the page banner say when the greatest rank belongs to a file of a different kind than the rest?

5. **Token age per tree.** Should a policy be able to supply a maximum token age, so a sensitive folder demands a token issued within the hour while the rest accept a day's?

6. **Tightening across a partition.** Should some classifications make policy changes wait for a quorum, so that a partitioned island cannot honour a policy that was tightened elsewhere?

7. **Break glass.** Medical systems let a clinician open a record they are not normally allowed to, in an emergency, on condition that it is recorded and reviewed. Is that a `read` granted with an audit record, and what does the banner say?

8. **Erasure.** GDPR gives a person the right to have their data erased, and this design never loses a version and replicates it everywhere. Erasing needs a tombstone that every shard honours by deleting the blobs of the named versions. Is that in scope?

7.3 Replication and storage

1. **The same path written on both islands.** During a partition two people can upload `plans.doc` into one folder, one on each island. They get different `id`s, so after repair one path names two files. Is the later one renamed, are both shown, or does one become a version of the other? Folders, including a seat's home folder made the first time its token is seen, have the same problem, and one proposed answer: derive a folder's `id` from its path, so that `mkdir -p` on both islands makes the same folder, and two first policies become concurrent versions of one policy, which both must allow, and so fail safe.

2. **Clocks.** `dt` is nanoseconds at the origin, and the latest version is the greatest `(dt, shard)`, so an island whose clock runs fast wins every concurrent edit. Is that acceptable, or should `dt` be a hybrid logical clock that never runs behind anything its shard has seen?

3. **Space.** Nothing is ever lost, so superseded and deleted versions keep their blobs. Is anything ever reclaimed, and under whose policy?

4. **Volume.** Many cameras, all day: a record is visible only once its blob is here, so every shard that replicates a camera folder holds all of its video. Should a folder's policy be able to keep its raw video at its origin while the derived text replicates, with the video fetched by range on demand? That relaxes "a visible record has its blob" for those folders.

5. **A new shard.** Its cursors start at zero, and it pulls every peer's whole feed; the waiting bound of 100000 records throttles that. Should a new shard start from a copy instead?

6. **A seat's home shard.** Does a seat write only on its home shard, which would make concurrent edits of its files rare, or anywhere?

7. **Retried uploads.** How long is `request_id` remembered, and is a retried upload on the other island a second file (as specified) or the same one?

7.4 Content, jobs and feeds

1. **Uploading a folder.** With `curl`, one file at a time in parts, or a tar that the shard unpacks, each entry passing the same checks?

2. **The job chain.** Which jobs run on which kinds of file, and may a derived file trigger further jobs (`x.mp4--audio.wav--lyrics.vtt`)? Jobs run only where the blob was created, so the chain runs once.

3. **Where jobs run.** LLM jobs want GPUs, which may be one big machine per island. May a worker sit on another machine in the island, and how does it reach the blob and return its outputs?

4. **Which models may see a file?** A folder's policy could supply the models, or model origins, that jobs may run on its files (an earlier map system had a rule against Chinese models); a job whose model is not allowed does not run. Per file, per job, or both?

5. **New questions of old footage.** After an event, people want new jobs run over weeks of video. Can jobs be requested again later, over a range of old files, and bounded so they do not starve the live cameras?

6. **Annotations more sensitive than their source.** An annotation that names a suspect gets the video's permissions by default; its own `--policy.rego` can narrow them. Should some jobs write their outputs with a narrower policy from the start?

7. **Cameras never finish.** A file exists only when it is complete, and a camera's stream never is. Does a camera upload a segment a minute, each its own file, with a wall-clock start in a clock the cameras agree on?

8. **Moving objects.** A track that reports every second is a new version every second. Is a track a file whose versions are its positions, or a series of segment files? Either way the map needs the latest position of thousands of objects at once.

9. **Feeds that cost money.** Every paid external feed needs a budget it cannot exceed: a cap on requests or spend per period, after which it stops and says so. Who sets it, per feed or per island?

7.5 Search and speed

1. **Search is a service.** Indexed things live in a database behind an API, so search is mounted under `/apis/`, filled by subscribing to the shard's feed. A hit's name needs `list`; its snippet and position (seconds into a video) need `read`; a count of hidden hits must not show. Which engine: Elasticsearch, as the original should have used, or something embedded?

2. **How fast is a query?** A listing, a feed page, a search: what is the target for the first result, under what load? An earlier map system's ten seconds is the number to beat by a wide margin.

7.6 Subscribers

A UI subscribes to a shard's feed exactly as a peer does. There is one feed: the records readable on that shard, in the order they became readable, read from a cursor in that shard's clock. A peer is the subscriber whose token can read everything; a browser is one whose token reads less. Replication is invisible to the UI, because an import is one more event in the feed. Two things follow for free:

- **Failover is a second cursor.** A browser keeps a cursor per shard, {`a: 50, b: 99`}, exactly as a shard keeps one per peer. Moving to the other island resumes from that island's cursor, and a record seen on both is dropped by its `(id, dt)`.

- **Order is the same function.** Events arrive in commit order, so a concurrent version can arrive after a newer one. The UI keeps the greatest `(dt, shard)` per `id`, the same function every shard computes.

What comes only from a token that cannot read everything (a policy change reaching back in time, and a cursor that reveals hidden activity) is in the security questions above.

7.7 Agreed, not yet in Lean

- **The tree and its policies.** Content at ordinary URLs under `/files/<org>/<seat>`; metadata built as a table from the root down, each folder's `.--policy.rego` and then the file's `<name>--policy.rego` overwriting the entries they set; a seat's home folder and its policy made on the first verified token. Writing into an empty folder may supply its `.--policy.rego` first; otherwise the folder gets its parent's. Writing a file creates the missing folders on its path, like `mkdir -p`, if the walk to the nearest existing folder allows it. The specification still uses `plans.pdf.rego` and `/files/{id}/{dt}/content`.
- **`list` beside `read` and `write`.** 404 without `list`, 403 with `list` but not `read`; `list` narrows down the path and `read` implies it. The `Decision` message in `Requirements.proto` has no `list` yet, and `AccessDenied` in `Session.lean` (`Download`, `Forward`, `Socket`) allows only 401 and 404, not 403.
- **Names are content.** A name the caller may not list never reaches them, and every channel that could carry one needs a negative requirement and a test:
  - listings, feed pages, history and search results;
  - a 404 body or header that echoes the requested path;
  - server and access logs, which must record ids rather than paths, or be protected at the level of what they record;
  - the `Referer` header a viewer frame sends, and browser history: content pages are served with `Referrer-Policy: no-referrer`;
  - metadata and job outputs, whose names embed their source's name: by default they share its permissions, and a policy may not loosen that;
  - the paths of folders between the root and a readable file.

  A caller who already knows a hidden path learns nothing from asking for it: the answer is the same as for a path that never existed.
- **No classification system in the source.** The server never names a marking, a rank or a rule; they all come from policy files. Tests: the same binary serves a NATO-marked tree and a GDPR tree, each with the right banners and decisions, and the source contains no marking strings.
- **Trust is chosen per shard.** Each shard picks its peers and its token issuers. The properties "Converge after repair" and "Same view everywhere after replication" in `Requirements.lean` still speak of every shard; they must be restated per token: any two shards that accept the token, and hold the same records, give it the same answers; and the shards that accept a token converge with each other after repair.
- **The door.** `ingressHttps` serves `/files/` and forwards `/apis/` mounts to internal services. Its protocols, `Forward` and `Socket`, are in `Session.lean` and checked; its negative requirements are not yet: no service is reachable except through it; no request reaches a service without a verified token and an allowing walk; no identity header from a client reaches a service; every request is counted per mount.

- **Apps read, APIs write.** An uploaded app is read-only to its users. Every API that writes identifies the user and checks their permissions on every call, with negative requirements and tests like the core: no call without a verified identity; no write the policy at its path does not allow; no input reaching a query, a shell or a path unescaped; no answer that names what the caller may not list; every write recorded with who made it.
- **Two halves, one pattern.** `/files/` for data and apps, `/apis/` for services users mount. An API serves its data as a feed read by cursors, with no state per user, like the record feed.
- **One feed for browsers and peers.** `FeedRequest` and `RecordsRequest` become one message: records past a cursor, in the shard's `visible_dt` order, filtered by the caller's token. The two tasks, and their properties, are still separate in `Requirements.lean`.
- **Content brings its viewer.** The shard serves uploaded HTML, scripts and styles as a web site, so a folder can be an app; deep links such as `?ts=40` are the viewer's own convention.
- **Derived files are named `source--x`.** With two dashes they are metadata about their source and share its permissions by the naming convention. `Requirements.lean` still names them with one dash.
- **Every derived thing answers "what is this?".** A report entry, a map object or a search hit links back, through its name, to the raw file it came from, which the caller may open only if they may read it.
- **Replay costs no blob.** A resent record already held, by its `(id, dt)`, causes no `BlobRequest`. It is in `Implementation.lean`, but not yet a requirement a test can check.
- **Seeking is fast everywhere.** The first byte of any range arrives within a fixed time, whatever its offset in the file: the 9000 MB video that seeked instantly. It is not yet a performance requirement.
- **Simulated sources for stress tests.** Like the fake airport that made planes fly around an earlier map: simulated cameras, feeds and tracks, at rates set by the test, drive the performance requirements (uploads per second, the latest positions of thousands of tracks, the first byte of a search) on both islands, through a partition and its repair.

7.8 Limits of the tools

- The models in `Islands.lean` are written by hand; they are not yet derived from the session types.
- The MCP server refuses arguments over 64 KiB (`maxArgumentBytes` in LeanFM's `Logic.lean`; HTTP itself allows 8 MiB) and models over 4096 nodes. Every model here is scaled down to fit: two shards, two or three records, two lanes. `repl-cursor` is 61 KB. A larger bound with a time budget would let the models grow.
- A nested `EF` witness stops at its first conjunct. `mcp_islands.py` joins the rest by asking again from where it stopped.
- Liveness holds only under fair delivery. The server shows the lassos where it fails, and the requirements state the assumption instead of hiding it.

Appendix A

The project Makefile

The project needs this file at `projects/IslandUploads/Makefile`, and `book/Makefile` is a copy of PaxosKV's. Copying from a PDF turns tabs into spaces, so recipe lines are written with `>` and the last line turns them into tabs. From the LeanFM root:

```
cp projects/PaxosKV/book/Makefile projects/IslandUploads/book/Makefile
cat > projects/IslandUploads/Makefile <<'MAKEFILE'
LEANFM := $(abspath ../..)
.PHONY: all check spec-check session models mcp book clean
all: check book
spec-check:
> cd $(LEANFM) && python3 scripts/chatui_specification.py $(CURDIR) \
    Project.Requirements.requirements Project.Implementation.implementation
session:
> cd $(LEANFM) && lake build LeanFM.Session && \
    lake env lean $(CURDIR)/spec/Session.lean
models:
> cd $(LEANFM) && lake env lean --run $(CURDIR)/spec/Islands.lean
mcp: models
> python3 scripts/mcp_islands.py
> python3 scripts/mcp_report.py
check: spec-check session
book:
> $(MAKE) -C book
clean:
> $(MAKE) -C book clean
MAKEFILE
perl -pi -e 's/^> /\t/' projects/IslandUploads/Makefile
```